

BeautifulSoup Documentation Analytics System

Project type: Final Project - Python Data Engineering & Analytics Pipeline

Target source: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

Stack: Python 3.11–3.13 · FastAPI · Streamlit · BeautifulSoup4 · Pandas · NumPy · Matplotlib · scikit-learn · nltk

Table of Contents

1. [Project Overview](#)
 2. [Team & Responsibility Matrix](#)
 3. [Architecture & Design Decisions](#)
 4. [Repository Structure](#)
 5. [Pipeline - ETL Stages](#)
 6. [Analytical Questions & Methods](#)
 7. [Visualizations](#)
 8. [Advanced NLP Layer](#)
 9. [REST API Reference](#)
 10. [Streamlit Frontend](#)
 11. [Report Generation](#)
 12. [Test Suite](#)
 13. [Configuration & Environment](#)
 14. [Quick-start Guide](#)
 15. [Known Limitations & Design Trade-offs](#)
-

1. Project Overview

This system is an **end-to-end data engineering and analytics pipeline** that automatically:

1. **Collects** the BeautifulSoup 4 official documentation from the web via HTTP
2. **Parses** the raw HTML into a structured section tree using BeautifulSoup4 itself
3. **Extracts** three structured datasets: documentation sections, hyperlinks, and Python code examples
4. **Analyses** the extracted data across 10 analytical questions using Pandas and NumPy
5. **Visualises** patterns with 8 charts (4 required + 4 advanced NLP-derived)
6. **Generates** a Markdown analytical report and a 5-sheet Excel workbook
7. **Serves** all processed data through a FastAPI REST API with 7 route prefixes and 20+ endpoints
8. **Presents** all findings through an interactive 7-page Streamlit web dashboard
9. **Applies** advanced NLP techniques: TF-IDF keyword ranking, Flesch-Kincaid readability scoring, cosine similarity between sections, and nltk for rich stopwords set

The project was designed to demonstrate modular, production-grade data pipeline architecture where each stage is independently runnable, testable, and maintainable.

2. Team & Responsibility Matrix

Member	Pipeline ownership	API route ownership	Frontend ownership	Required features
Dat (B)	<code>collector.py</code> , <code>parser.py</code>	<code>api/main.py</code> , <code>data_service.py</code> , <code>/sections</code>	-	F1, F2
Phuc (C)	<code>extractor.py</code> , <code>link_classifier.py</code>	<code>/links</code>	<code>app/main.py</code> , pages 1–3	F3, F4
Duong (D)	<code>analyzer.py</code> , <code>visualizer.py</code>	<code>/analytics</code>	<code>tests/</code> , <code>config/</code>	F6, F7
Hung (A)	<code>code_extractor.py</code> , <code>reporter.py</code> , <code>pipeline.py</code> , <code>advanced/</code>	<code>/code</code> , <code>/pipeline</code> , <code>/search</code> , <code>/graph</code>	pages 0, 4, 5, <code>components/</code>	F5, F8 + advanced

Route ownership principle: whoever writes the pipeline stage that produces a dataset also writes the API route that serves it. This enforces end-to-end ownership and prevents knowledge gaps between data production and data serving.

Shared contracts (Hung, Day 1): `shared/schemas.py`, `shared/constants.py`, and `shared/utils.py` were committed to `main` before any other member branched. These define all Pydantic models, file paths, column names, and utility functions - every other file imports from here, never hardcodes values directly.

3. Architecture & Design Decisions

3.1 Why FastAPI + Streamlit (not Tkinter/Flask)

The project uses a **two-service architecture**: FastAPI (port 8001) as a REST backend and Streamlit (port 8501) as the frontend. This was chosen over the simpler alternatives because:

- **Separation of concerns:** the pipeline, data serving, and presentation layers are completely decoupled. The pipeline writes CSVs; the API reads them; the frontend calls the API. No layer knows about the implementation of another.
- **FastAPI auto-generates Swagger UI** at `/docs`, providing a self-documenting, testable API surface without any additional work.
- **Streamlit** enables fully interactive Python-native dashboards without writing any HTML, CSS, or JavaScript.
- **Testability:** the API can be tested with `TestClient` without running a real server. The pipeline stages can be tested by feeding DataFrames directly. Neither requires the other to be running.
- **Demo quality:** the combination produces a visually impressive, interactive application that demonstrates professional-grade data engineering.

3.2 Why a 6-stage sequential pipeline with an orchestrator

The pipeline is broken into 6 discrete, independently-runnable stages:

```
collector → parser → extractor + code_extractor → analyzer → visualizer → reporter
```

Each stage:

- Has a single, well-defined input (file path or in-memory object) and output (file path or return value)
- Can be run standalone via `python -m pipeline.<stage_name>`
- Prints human-readable progress to stdout with timing
- Is testable in isolation with synthetic inputs

The pipeline exposes live progress through two mechanisms. The primary path is a **WebSocket endpoint** (`WS /ws/pipeline-progress`) that spawns `pipeline.pipeline` as a subprocess using `sys.executable` (ensuring the correct venv is used) and streams each stdout line to the Streamlit frontend via `websocket-client`. A **toggle on the Home page** switches to HTTP polling (`GET /pipeline/status` every 1 second) as a fallback - this activates automatically if `websocket-client` is not installed or if the WebSocket connection fails. The `ConnectionResetError [WinError 10054]` that Windows raises when a client disconnects abruptly is caught in a dedicated `except` block and logged only at DEBUG level, eliminating the cosmetic error from production logs.

The `--skip-fetch` flag re-uses the cached HTML file, enabling fast iteration during development without hitting the network on every run.

3.3 Shared contracts pattern

All data shapes are defined once in `shared/schemas.py` as Pydantic models. All file paths and column names are defined once in `shared/constants.py`. This means:

- A column rename requires changing exactly one file
- Pydantic validates every API response automatically - schema drift is caught at the route level, not at the frontend
- Tests can assert against `SECTION_COLS`, `LINK_COLS`, `CODE_COLS` without hardcoding column names

3.4 Real-data testing approach

Tests run against **real pipeline output**, not mock fixtures for most cases. A `@skip_if_no_data` decorator skips data-dependent tests if the pipeline hasn't been run yet, then all tests pass once it has. This makes the test suite both development-friendly (no setup required to run the non-data tests) and meaningful (assertions verify real pipeline behaviour, not synthetic toy cases).

The only exceptions are `test_analyzer.py`, `test_extractor.py`, `test_parser.py`, and `test_collector.py`, which use small synthetic inputs so they can run offline without pipeline output.

For `test_api_routes.py`, the fixture pre-populates `data_service._cache` directly rather than patching `load_*` functions. This is necessary because route modules import `load_sections`, `load_links`, etc. **by reference** at import time - patching the `data_service` namespace after that has no effect on the already-bound local names in each route module.

3.5 Fuzzy search (Levenshtein) - Sections and Code pages

Two pages implement fuzzy search using `rapidfuzz` - a fast Cython-compiled implementation of Levenshtein distance algorithms:

Sections page (`1_sections.py`): searches against section title and first 300 characters of section text.

Code page (`3_code.py`): searches against `section_title` of each code example, enabling queries like "searching tree" to find all code examples from the "Searching the tree" section regardless of exact wording.

Both pages use the same two-function scoring strategy:

- `fuzz.partial_ratio` - handles substring matching ("find all" finds "find_all() arguments")
- `fuzz.token_sort_ratio` - handles word-order differences ("tree searching" finds "Searching the tree")

The best score across both functions is taken as the match score. A sensitivity slider (30–95, default 60 on the code page and 55 on the sections page) lets the user tune the threshold interactively. Each matching result shows its match score as a badge (e.g. **87%**) so the user understands why it was included.

The fuzzy filter runs **client-side** on the full list of fetched items rather than passing a filter parameter to the API. This is intentional: the API's

`section` parameter performs exact string matching, which would defeat the purpose of fuzzy search. Fetching all items and filtering in the browser costs negligible time given the dataset size (~220 code examples, ~113 sections).

Both implementations include a graceful fallback to plain `str.contains()` substring matching if `rapidfuzz` is not installed, so the pages remain functional even without the dependency.

4. Repository Structure

```
bs4_analytics/  
├── shared/                # Contract package - Hung owns, everyone  
├── imports  
│   ├── schemas.py        # Pydantic models: SectionModel, LinkModel,  
│   │                     # CodeExampleModel, AnalyticsSummary, + API  
│   └── wrappers  
│       ├── constants.py  # All file paths, URLs, column lists,  
│       │                 # chart paths, STOPWORDS - never hardcoded  
│       └── LINK_TYPES,  
│           elsewhere  
│               ├── utils.py    # load_sections/links/code_examples(),  
│               │               # pipeline_has_run(), data_files_status(),  
│               └── ensure_dirs(),  
│               word_count()  
└── pipeline/              # ETL stages - one file per stage
```

```

|   |   | collector.py           # F1: HTTP GET → BeautifulSoup_doc.html
|   |   | parser.py             # F2: HTML → BeautifulSoup object → section
tree
|   |   | extractor.py          # F3+F4: soup → sections.csv + links.csv
|   |   | link_classifier.py     # F4: classify_link(href) → one of 5
LINK_TYPES
|   |   | code_extractor.py      # F5: soup → code_examples.csv
|   |   | analyzer.py           # F6: 3 CSVs → 10 answers →
summary_stats.json
|   |   | visualizer.py         # F7: 8 charts → output/charts/*.png
|   |   | reporter.py           # F8: Markdown report + Excel workbook
|   |   | pipeline.py           # Orchestrator: runs all stages in order
|   |   |   | advanced/
|   |   |   | | nlp_analyzer.py  # TF-IDF ranking, Flesch readability, nltk
|   |   |   | | similarity.py    # Cosine similarity matrix between sections
|   |   |   | | graph_builder.py # networkx section-link graph → D3 JSON
|   |   |
|   |   | api/                  # FastAPI backend - port 8001
|   |   |   | main.py           # App init, CORS, static mount, router
wiring
|   |   |   | routes/
|   |   |   | | sections.py      # GET /sections, GET /sections/{id}
|   |   |   | | links.py        # GET /links, GET /links/stats
|   |   |   | | code.py         # GET /code-examples, GET /code-
examples/{id}
|   |   |   | | analytics.py     # GET /analytics/summary, /charts, /link-
types
|   |   |   | | pipeline.py      # POST /pipeline/run, GET /pipeline/status
|   |   |   | | search.py        # GET /search?q=&target=
|   |   |   | | graph.py        # GET /graph
|   |   |   | | services/
|   |   |   | |   | data_service.py # In-memory DataFrame cache; all routes
import from here
|   |   |   | | websocket.py     # WS /ws/pipeline-progress (advanced)
|   |   |   |
|   |   |   | app/              # Streamlit frontend - port 8501
|   |   |   |   | main.py        # Entry: set_page_config, sidebar, API
health check
|   |   |   |   | config.py      # Reads .env directly → API_BASE, WS_BASE
|   |   |   |   | pages/
|   |   |   |   | | 0_home.py    # Pipeline trigger, polling progress, stat
cards
|   |   |   |   | | 1_sections.py # Fuzzy section search (rapidfuzz
Levenshtein)
|   |   |   |   | | 2_links.py    # Link explorer, pie chart, URL resolution
|   |   |   |   | | 3_code.py     # Code browser, method filter, syntax
highlight
|   |   |   |   | | 4_analytics.py # 8-tab Plotly dashboard, heatmap,
wordcloud, graph
|   |   |   |   | | 5_report.py    # Download MD report, XLSX, chart PNGs
|   |   |   |   | | components/
|   |   |   |   | | | stat_cards.py # Reusable st.metric rows
|   |   |   |   | | | wordcloud_widget.py # wordcloud library → st.image

```

```

|       └─ network_graph.py           # pyvis interactive section dependency
graph
|
|─ tests/
|   └─ test_collector.py               # 6 tests - mocked HTTP, UTF-8, directory
creation
|   └─ test_parser.py                 # 9 tests - inline HTML fixture, section
tree validation
|   └─ test_link_classifier.py         # 25 parametrized cases - all 5 branches +
edge cases
|   └─ test_extractor.py               # 12 tests - monkeypatched CSV paths, 5
link types
|   └─ test_analyzer.py               # 12 tests - synthetic DataFrames, Q1-Q10
exact values
|   └─ test_api_routes.py              # 28 tests - TestClient, cache pre-
population strategy
|   └─ test_pipeline_output.py         # 28 tests - real pipeline data,
@skip_if_no_data guard
|
|─ config/
|   └─ settings.py                    # Pydantic BaseSettings - reads from .env
|   └─ logging.py                     # dictConfig - colored console in dev
|
|─ notebooks/
|   └─ analysis.ipynb                 # 15 cells: Q1-Q10 + 3 advanced +
findings/limitations
|
|─ data/                               # gitignored - pipeline-generated
|   └─ raw/beautifulsoup_doc.html
|       └─ processed/
|           └─ sections.csv
|           └─ links.csv
|           └─ code_examples.csv
|           └─ summary_stats.json
|
|─ output/                             # gitignored - pipeline-generated
|   └─ charts/                         # 8 PNG files
|       └─ report/
|           └─ final_report.md
|           └─ summary_tables.xlsx
|
|─ docker/
|   └─ Dockerfile.api
|   └─ Dockerfile.app
|   └─ docker-compose.yml
|
|─ .github/workflows/ci.yml            # pytest + ruff on every PR to main
|─ requirements.txt
|─ .env.example

```

5. Pipeline - ETL Stages

Stage 1 - Collector (`pipeline/collector.py`) · Feature F1

Owner: Dat

Sends a single HTTP GET request to `TARGET_URL` using the `requests` library with a 20-second timeout and a descriptive User-Agent header. Calls `response.raise_for_status()` to automatically raise `HTTPError` on any non-200 response. Writes the response body to `data/raw/beautifulsoup_doc.html` in UTF-8 encoding.

The destination's parent directory is created with `dest.parent.mkdir(parents=True, exist_ok=True)` before writing, so the stage is safe to run in a clean environment.

Design decision - save before parse: the raw HTML is saved to disk before any parsing occurs. This means:

- Subsequent pipeline runs can use `--skip-fetch` to skip the network request
- The raw source is preserved for auditing and reproducibility
- Parser bugs don't require re-downloading

```
Input:  TARGET_URL = "https://www.crummy.com/software/BeautifulSoup/bs4/doc/"
Output: data/raw/beautifulsoup_doc.html (~500 KB)
```

Stage 2 - Parser (`pipeline/parser.py`) · Feature F2

Owner: Dat

Loads the saved HTML with `BeautifulSoup(html, "html.parser")` - the pure-Python parser bundled with Python's standard library, chosen for zero external C dependencies. The `lxml` parser is faster but requires a compiled binary; `html.parser` is always available and sufficient for well-formed Sphinx-generated HTML.

`extract_section_tree()` algorithm:

The function iterates all `h1`, `h2`, `h3` tags in document order using `soup.find_all(["h1", "h2", "h3"])`. For each heading:

1. The heading level is extracted from `tag.name` ("`h1`" → 1, etc.)
2. The heading title is obtained with `heading.get_text(strip=True).replace("¶", "").strip()` - the ¶ pilcrow symbol is a permalink anchor Sphinx inserts next to every heading; it must be stripped for clean titles
3. The content is collected by iterating `.next_siblings` and stopping when a sibling is a heading tag with level ≤ current level
4. All sibling text is joined and cleaned with `re.sub(r"\s+", " ", ...)` to normalise whitespace
5. `word_count` is computed by splitting on whitespace
6. `code_block_count` is computed by counting `<div class="highlight">` descendants

7. `link_count` is computed by counting `<a>` descendants

Design decision - section boundaries by heading level: this correctly handles nested sections (H1 → H2 → H3) without requiring knowledge of the document structure in advance. A H2 heading ends when the next H1 or H2 is encountered.

```
Input: data/raw/beautifulsoup_doc.html
Output: BeautifulSoup object (in-memory), list[dict] (returned to extractor)
```

Stage 3 - Extractor (`pipeline/extractor.py`) · Features F3 & F4

Owner: Phuc

`extract_sections()` calls `extract_section_tree()` from the parser, wraps the result into a DataFrame, and writes `sections.csv` with columns matching `SECTION_COLS` exactly.

`extract_links()` iterates `soup.find_all("a")` and for each tag:

- Extracts `href` with `tag.get("href", "")` or `""`
- Strips ¶ from link text (Sphinx adds pilcrow anchors as `<a>` tags)
- Skips links where the stripped text is empty and the href is a bare anchor (pilcrow permalink anchors)
- Calls `classify_link(href)` to determine the link type
- Calls `_find_parent_section()` to determine which section contains this link

`_find_parent_section()` algorithm: uses `tag.find_all_previous(["h1", "h2", "h3"])` to find the nearest preceding heading in document order, then looks up the heading text in the sections DataFrame. This is correct for Sphinx-generated HTML where headings are preceding siblings, not ancestors - using `.parents` would fail because the heading is not an ancestor of the content following it.

```
Input: BeautifulSoup object, sections DataFrame
Output: data/processed/sections.csv (7 columns), data/processed/links.csv (4 columns)
```

Stage 3b - Link Classifier (`pipeline/link_classifier.py`) · Feature F4

Owner: Phuc

A pure function with no side effects, no I/O, and no dependencies beyond `shared.constants`. Classifies any href string into exactly one of five canonical types using a priority-ordered rule chain (first match wins):

Priority	Type	Rule
----------	------	------

Priority	Type	Rule
1	<code>empty_or_invalid</code>	<code>None</code> , <code>"</code> , <code>"#"</code> , any <code>javascript:</code> prefix
2	<code>image_link</code>	href ends with <code>.jpg</code> , <code>.jpeg</code> , <code>.png</code> , <code>.gif</code> , <code>.svg</code> , <code>.webp</code> , <code>.ico</code> , <code>.bmp</code>
3	<code>internal_anchor</code>	starts with <code>"#"</code> (but not bare <code>"#"</code> , already caught)
4	<code>documentation_link</code>	starts with <code>TARGET_URL</code> or contains <code>"crummy.com"</code>
5	<code>external_link</code>	all other absolute URLs

Design decision - pure function: making `classify_link()` a pure function with no state means it can be independently unit-tested with 25 parametrized test cases covering every branch and edge case, without any setup.

Stage 4 - Code Extractor (`pipeline/code_extractor.py`) · Feature F5

Owner: Hung

Sphinx-generated documentation wraps every Python code block in `<div class="highlight">`. The extractor finds all such blocks with `soup.find_all("div", class_="highlight")` and for each:

- Extracts code text with `block.get_text()`
- Counts non-empty lines: `len([l for l in code_text.splitlines() if l.strip()])`
- Finds the parent section with `tag.find_all_previous(["h1", "h2", "h3"])` (same strategy as link extractor)
- Sets 5 boolean flags by checking whether key method strings appear in the code text:

Column	Detected by
<code>contains_find_all</code>	<code>"find_all"</code> in <code>code_text</code>
<code>contains_find</code>	<code>".find("</code> in <code>code_text</code>
<code>contains_select</code>	<code>".select("</code> in <code>code_text</code>
<code>contains_get_text</code>	<code>"get_text"</code> in <code>code_text</code>
<code>contains_requests</code>	<code>"requests"</code> in <code>code_text</code>

Design decision - string presence detection: simple `in` checks are used rather than regex or AST parsing. This is intentional: the goal is to flag which examples demonstrate a method, not to parse Python syntax. The string `.find(` (with the dot) avoids false positives from plain English words like "find".

Input: BeautifulSoup object
Output: data/processed/code_examples.csv (9 columns)

Stage 5 - Analyzer (`pipeline/analyzer.py`) · Feature F6

Owner: Duong

Reads all three processed CSVs and computes 10 analytical questions. Results are saved to `summary_stats.json` and returned as a Python dict.

Question	Method
Q1 - Total sections	<code>len(sections)</code>
Q2 - Highest word count section	<code>sections.loc[sections["word_count"].idxmax(), "section_title"]</code>
Q3 - Most code examples section	<code>code.groupby("section_title").size().idxmax()</code>
Q4 - Most links section	<code>links.groupby("section_title").size().idxmax()</code>
Q5 - Top 10 keywords	Regex <code>\b[a-z][a-z0-9_]{2,}\b</code> , Counter, STOPWORDS filter
Q6 - Link type counts	<code>links["link_type"].value_counts().to_dict()</code>
Q7 - <code>find_all()</code> count	<code>int(code["contains_find_all"].sum())</code>
Q8 - <code>get_text()</code> count	<code>int(code["contains_get_text"].sum())</code>
Q9 - Average words/section	<code>round(float(sections["word_count"].mean()), 2)</code>
Q10 - Sections with no code	<code>int((sections["code_block_count"] == 0).sum())</code>

Q5 keyword extraction detail: the regex `\b[a-z][a-z0-9_]{2,}\b` captures words starting with a letter, followed by at least 2 alphanumeric/underscore characters. This captures Python identifiers like `find_all` and `beautifulsoup` while excluding short words. A 68-word STOPWORDS set defined in `shared/constants.py` filters out common English words. The result is a Counter of the remaining terms.

Advanced analytics - after the core pipeline runs, `_stage_advanced()` is called as a 7th stage. It adds three fields to `summary_stats.json` under `adv_` prefixed keys:

- `adv_top_tfidf_keywords` - TF-IDF weighted keyword list (list of `{keyword, count}`)
- `adv_avg_readability_score` - mean Flesch Reading Ease across all sections (float)
- `adv_top_similar_pairs` - top 10 most similar section pairs (list of `[title_a, title_b, score]`)

These fields default to `null` in `AnalyticsSummary` so existing consumers never break if the advanced stage hasn't run.

```
Input: sections.csv, links.csv, code_examples.csv
Output: data/processed/summary_stats.json
```

Stage 6 - Visualizer (`pipeline/visualizer.py`) · Feature F7

Owner: Duong

Generates 8 PNG charts to `output/charts/`. Uses `matplotlib.use("Agg")` for headless server-side rendering (no display required). All charts use a consistent style (`seaborn-v0_8-whitegrid`), color palette (`#4C72B0` primary), and 150 DPI resolution.

4 required charts:

Chart	Type	Key design choices
Word count by section	Horizontal bar	Top 10, sorted descending, value labels on bars, truncated titles
Code examples by section	Vertical bar	Top 10, rotated 40° x-labels, value labels above bars
Link type distribution	Pie	5 wedges, no inline labels (too crowded), legend with counts
Code line count histogram	Histogram	20 bins, mean/median vertical lines, stats annotation box

4 advanced charts (require scikit-learn / textstat):

Chart	Type	Data source
TF-IDF vs raw frequency	Side-by-side horizontal bar	<code>TfidfVectorizer</code> from scikit-learn
Readability by section	Color-coded horizontal bar	<code>textstat.flesch_reading_ease()</code> per section
Similarity heatmap	Imshow heatmap	<code>cosine_similarity</code> on TF-IDF vectors, top 30 sections
Method usage	Grouped bar (count + %)	Boolean column sums from <code>code_examples.csv</code>

The advanced charts gracefully degrade - if scikit-learn or textstat are not installed, a placeholder image is saved with an installation message.

Stage 7 - Reporter (`pipeline/reporter.py`) · Feature F8

Owner: Hung

Generates two artifacts:

`final_report.md` - A structured Markdown document with 8 sections matching the required report structure: Dataset Overview, Scraping Method, Extracted Data Summary, Analysis Results (Q1–Q10 table), Charts (HTTP URL references to API-served PNGs), Key Findings, Limitations, and Conclusion. Charts are

embedded as `![name](http://localhost:8001/static/charts/filename.png)` - this works in any Markdown renderer while the API is running.

summary_tables.xlsx - A 5-sheet Excel workbook built with `openpyxl` via `pd.ExcelWriter`:

- Sheet 1: **Sections** - full sections DataFrame
- Sheet 2: **Links** - full links DataFrame
- Sheet 3: **Code Examples** - full code_examples DataFrame
- Sheet 4: **Analytics Summary** - Metric/Value pairs for all 10 questions plus advanced fields
- Sheet 5: **Top Keywords** - keyword/count pairs from Q5

6. Analytical Questions & Methods

Results from a representative pipeline run (verified against generated charts):

#	Question	Result
Q1	Total sections	113
Q2	Highest word count section	"Searching the tree" - 4,711 words
Q3	Most code examples section	"Multi-valued attributes" - 13 examples
Q4	Most links section	"Table of Contents" - 128 links
Q5	Top 10 keywords (NLTK-filtered raw)	soup (926), tag (746), class (511), html (420), example (419), string (387), com (383), href (378), http (363), beautiful (351)
Q6	Link type breakdown	internal_anchor 342 (91.0%), external_link 21 (5.6%), documentation_link 11 (2.9%), empty_or_invalid 2 (0.5%), image_link 0
Q7	find_all() examples	41 (18.6% of 220 total)
Q8	get_text() examples	4 (1.8%)
Q9	Average words/section	363.28
Q10	Sections with no code	22
ADV	Mean Flesch readability	56.0 (Standard band: 50–60)
ADV	TF-IDF top term	tag (13.30), narrowly above soup (13.26)
ADV	Most similar pair	"Low-level search interface" ↔ "Custom element filtering"

7. Visualizations

Required (F7)

Chart 1 - Top Sections by Word Count ([word_count_by_section.png](#))

Horizontal bar chart of the top 10 sections by word count, sorted descending, with value labels on each bar. "Searching the tree" leads at 4,711 words - more than twice "Navigating the tree" (2,104) and nearly 13× the overall median section length. The top 3 sections (Searching the tree, Navigating the tree, find_all()) collectively contain the majority of the documentation's technical prose. Even the 10th-ranked section ("CSS selectors through the.cssproperty" at 1,055 words) is nearly 3× the mean, confirming the depth asymmetry is not a single outlier but a structural pattern across the most important topics.

Chart 2 - Code Examples by Section ([code_examples_by_section.png](#))

Vertical bar chart of the top 10 sections by code example count. "Multi-valued attributes" leads with 13 examples, followed by "CSS selectors through the.cssproperty" (12). Neither section appears in the top 10 by word count - example density and prose depth are independent dimensions. "The keyword arguments" (9), "Output formatters" (8), and "Searching by CSS class" (7) round out the top 5. Six sections tie at 5–6 examples. The code-dense sections are predominantly API-reference sections covering specific argument types, attribute matching, and CSS selector syntax - concepts that require repeated worked examples rather than extended explanation.

Chart 3 - Link Type Distribution ([link_type_distribution.png](#))

Pie chart with 4 visible wedges (image_link is absent - 0 detected at scrape time). [internal_anchor](#) dominates at 91.0% (342 of 376 total links). [external_link](#) accounts for 5.6% (21), [documentation_link](#) for 2.9% (11 links to other crummy.com pages), and [empty_or_invalid](#) for 0.5% (2). The near-total internal-anchor dominance means the documentation operates as a closed reference system: users are guided between sections of the same page rather than outward to external resources.

Chart 4 - Code Example Line Count Distribution ([code_linecount_hist.png](#))

Right-skewed histogram (n=220, mean=5.7, median=4.0, max=35, std=4.6). The modal bin is 3–4 lines (~63 examples at the peak), and most examples fall in the 1–6 line range. The mean–median gap of 1.7 lines confirms the rightward skew. A secondary mode around 8–10 lines represents intermediate examples. The long tail extends to 35 lines for the most complex multi-step demonstrations. The distribution reflects a deliberate pedagogical choice: BeautifulSoup's documentation favors short, self-contained code snippets over comprehensive program listings.

Advanced

Chart 5 - TF-IDF vs Raw Frequency ([tfidf_keywords.png](#))

Side-by-side horizontal bar chart. Raw frequency top 10: soup (926) → tag (746) → class (511) → html (420) → example (419) → string (387) → com (383) → href (378) → http (363) → beautiful (351). TF-IDF top 10: tag (13.30) → soup (13.26) → class (7.79) → html (7.47) → com (7.41) → example (7.38) → beautiful (7.23) → string (7.20) → title (6.64) → document (6.39).

[tag](#) overtakes [soup](#) in TF-IDF despite being second in raw frequency - [soup](#) is the ubiquitous variable name used in almost every code block, so it carries low discriminative power across sections. [href](#) and

`http` drop out of the TF-IDF top 10 entirely, as they appear uniformly across sections as part of example URLs. `title` and `document` enter the TF-IDF top 10 as more conceptually specific terms. Notably, `sister` - which dominated the top 10 under the previous hardcoded stopwords set - is absent from both rankings after NLTK filtering, confirming that the example-data artefact has been correctly suppressed.

Chart 6 - Readability by Section (`readability_by_section.png`)

Color-coded horizontal bar chart showing the 10 easiest (green >70) and 10 hardest (red <50) sections, with mean (56.0) and standard threshold (60) as vertical reference lines. Easiest: "True" (93.6) - a very short boolean-values section; "Generators" (89.4); "A regular expression" (81.7); ".string" (81.1); "A string" (78.8). These are concise definitional sections. Hardest real content: `find_previous_siblings()`... (21.1) and `find_next_siblings()`... (26.6) - dense API reference sections with long method signatures. "Table of Contents" and "Quick search" score 0.0 due to near-absence of prose sentences (degenerate formula output). "API documentation" scores 33.5 - the section describing BeautifulSoup's public API surface uses the most complex sentence structure. The mean of 56.0 is just below the 60 standard threshold, consistent with competent but demanding technical writing.

Chart 7 - Section Cosine Similarity Heatmap (`similarity_heatmap.png`)

Imshow heatmap of cosine similarity between the top 30 sections by word count (those with the most content to compare). Diagonal is 1.0 throughout. Notable off-diagonal clusters: "Searching the tree" ↔ "find_all()" shows one of the brightest off-diagonal cells, confirming that the parent search section and its most important subsection share nearly identical vocabulary. "Beautiful Soup 3" ↔ "Porting code to BS4" forms a bright pair - the migration guide reuses BS3 terminology throughout. "Output" ↔ "Output formatters" cluster together. The heatmap reveals a block structure: navigation sections (Searching, Navigating, Going sideways, Going back and forth) form one cluster; output sections another; installation sections another. Most off-diagonal cells are light blue (low similarity), confirming that despite shared vocabulary, most section pairs cover distinct content.

Chart 8 - Method Usage (`method_usage.png`)

Grouped bar chart with dual y-axes: absolute count (blue, left) and percentage of total (green, right). `find_all()`: 41 (18.6%) → `find()`: 14 (6.4%) → `select()`: 14 (6.4%) → `get_text()`: 4 (1.8%) → `requests`: 0 (0.0%). `find()` and `select()` tie at exactly 14 examples each - an unexpected result showing that the CSS selector interface (`select()`) receives equal demonstration depth as the traditional traversal method (`find()`). `requests` scoring 0 is significant: the documentation assumes users bring their own HTTP layer and makes no attempt to demonstrate data fetching, focusing entirely on parsing and navigation.

8. Advanced NLP Layer (scikit-learn · textstat · NLTK)

All advanced functionality is in `pipeline/advanced/` - additive files that never modify the core pipeline outputs.

8.1 TF-IDF Keyword Ranking (`nlp_analyzer.py`)

Uses `sklearn.feature_extraction.text.TfidfVectorizer` with `stop_words=list(STOPWORDS)` (198 NLTK English stopwords, same set used for Q5) and `max_features=500`. The vectorizer transforms all

section texts into a term–document matrix. Column sums give each term's total TF-IDF weight across the corpus.

Why TF-IDF over raw frequency: Raw frequency (Q5) counts total occurrences. TF-IDF weights a term by how distinctive it is to individual sections - a word appearing uniformly across all sections (like `soup`, the standard variable name) gets a low score even with high total frequency, while terms concentrated in specific sections score highly. This is confirmed by the chart: `tag` (13.30) overtakes `soup` (13.26) in TF-IDF despite being ranked second in raw frequency, and `href/http` drop out of the top 10 entirely as they appear uniformly across sections as URL components.

8.2 Readability Scoring (`nlp_analyzer.py`)

Uses `textstat.flesch_reading_ease()` per section. The Flesch formula:

$$\text{Score} = 206.835 - 1.015 \times (\text{words/sentences}) - 84.6 \times (\text{syllables/words})$$

Higher scores mean easier reading (0–100 scale). The mean score of 55.99 falls in the "Standard" band. Scores are computed per section, enabling comparison of introductory sections (high score, short sentences) vs. advanced sections (low score, long technical sentences).

Known limitation: `section_text` includes code snippets and Python identifiers, which have unusual syllable counts. This artificially deflates readability scores. A production system would strip code blocks before scoring.

8.3 Cosine Similarity (`similarity.py`)

TF-IDF vectors are computed for all sections, then `sklearn.metrics.pairwise.cosine_similarity` computes the full (n×n) pairwise similarity matrix. The upper triangle is enumerated to produce ranked section pairs.

Cosine similarity measures the angle between two vectors in the TF-IDF space - sections sharing many of the same weighted terms score close to 1.0. Sections with completely disjoint vocabularies score near 0.

8.4 Section-Link Network Graph (`graph_builder.py`)

Uses `networkx.DiGraph` to build a directed graph where nodes are sections and edges are `internal_anchor` or `documentation_link` type links between sections. Anchor slugs like `#searching-the-tree` are resolved to section titles using a three-pass matching strategy:

1. **Exact match** after normalising (lowercase, replace `-` with space)
2. **Substring match** (slug contained in title or vice versa)
3. **Word overlap** (at least 2 words in common)

The graph is exported as D3-compatible `{nodes, edges}` JSON served by `GET /graph` and visualised in Streamlit using `pyvis`.

9. REST API Reference

Base URL: <http://localhost:8001>

Auto-generated Swagger UI: <http://localhost:8001/docs>

Sections (</sections>) - Dat

Method	Endpoint	Description
GET	/sections/	Paginated list. Params: page , size , level (1–3), search
GET	/sections/{id}	Single section by ID. 404 if not found

All responses validated against `SectionModel`. Empty `section_text` cells from the CSV are filled with "" before Pydantic validation to prevent `ValidationError` on NaN values.

Links (</links>) - Phuc

Method	Endpoint	Description
GET	/links/	Filtered list. Params: link_type , section , search
GET	/links/stats	<code>{link_type: count}</code> for all 5 types. Zero-filled so all types always present

Code Examples (</code-examples>) - Hung

Method	Endpoint	Description
GET	/code-examples/	Filtered list. Params: method (one of 5 boolean column names), section
GET	/code-examples/{id}	Single example. 400 on invalid method name, 404 if ID not found

Boolean columns are explicitly cast to `bool` before serialisation to prevent pandas `True/False` from being serialised as integers.

Analytics (</analytics>) - Duong

Method	Endpoint	Description
GET	/analytics/summary	Loads from <code>summary_stats.json</code> - preserves <code>adv_</code> fields written by advanced pipeline
GET	/analytics/charts	<code>{display_name: "/static/charts/filename.png"}</code> for existing PNGs
GET	/analytics/link-types	Convenience alias for /links/stats

Design decision - load from JSON, not re-run: the analytics endpoint reads from `summary_stats.json` rather than re-running `run_all_analytics()` on every request. This is critical because

`run_all_analytics()` overwrites the JSON, which would wipe out the `adv_` fields written by the advanced pipeline stage. Loading the saved JSON preserves all fields.

Pipeline (`/pipeline`) - Hung

Method	Endpoint	Description
POST	<code>/pipeline/run</code>	Starts pipeline as FastAPI BackgroundTask. Returns 202 immediately
GET	<code>/pipeline/status</code>	<code>{running, current_stage, stages_done, total_stages, last_run, last_run_duration_seconds}</code>

The status endpoint enables Streamlit to poll progress at 1-second intervals without WebSockets. After each stage completes, `_state["stages_done"]` is incremented and `_state["current_stage"]` is updated - the Streamlit progress bar advances by `stages_done / total_stages`.

Search (`/search`) - Hung (advanced)

Method	Endpoint	Description
GET	<code>/search/</code>	Params: <code>q</code> (min 2 chars), <code>target</code> (sections/links/code/all). Returns <code>{query, target, total_matches, results}</code>

Graph (`/graph`) - Hung (advanced)

Method	Endpoint	Description
GET	<code>/graph/</code>	Returns D3-compatible <code>{nodes: [...], edges: [...]}</code> for section dependency network

Static files

Chart PNGs are served at `/static/charts/{filename}.png` via FastAPI's `StaticFiles` mount on the `output/charts/` directory. This allows the Markdown report and Streamlit pages to reference charts by URL.

10. Streamlit Frontend

All pages import API base URL from `app/config.py`, which reads `.env` directly without relying on `os.getenv()`. This ensures the correct port is always used regardless of how Streamlit is launched.

Page 0 - Home (`0_home.py`) · Hung

- **Dataset health:** calls `GET /health` and displays a 4-column metric grid showing which of the 4 processed data files exist
- **Pipeline trigger:** `POST /pipeline/run` starts the pipeline; the page then polls `GET /pipeline/status` every 1 second with `st.rerun()`, displaying a progress bar advancing by `stages_done / 7`
- **Quick stats:** metric cards from `GET /analytics/summary`

- **Last run info:** timestamp and duration from `GET /pipeline/status`

Page 1 - Sections (`1_sections.py`) · Phuc

- Fetches all sections (`size=200`) and filters client-side for fuzzy matching
- `rapidfuzz.fuzz.partial_ratio` + `token_sort_ratio` scored against both title and first 300 chars of text
- Each result shows a match score badge
- Long sections (>800 chars) have a Show more / Show less toggle using `st.session_state`
- Sensitivity slider (30–95) lets users tune without reloading

Page 2 - Links (`2_links.py`) · Phuc

- Plotly donut pie from `GET /links/stats`
- Internal anchor hrefs are prefixed with `https://www.crummy.com/software/BeautifulSoup/bs4/doc/` before display so clicking them navigates to the correct section of the real documentation
- `st.column_config.LinkColumn` for external URLs, `TextColumn` for internal anchors

Page 3 - Code (`3_code.py`) · Phuc

- Checkbox method filter (multi-select: first checked wins)
- `st.code(..., language="python")` for syntax-highlighted code display
- Method badge row showing which BS4 methods each example demonstrates

Page 4 - Analytics (`4_analytics.py`) · Hung

8-tab Plotly interactive dashboard:

1. Word Count - Plotly horizontal bar with hover tooltips
2. Code Examples - section bar + method usage grouped bar
3. Link Types - Plotly donut pie with percentage labels
4. Line Counts - Plotly histogram with mean/median vlines
5. Keywords (Frequency) - raw frequency bar
6. Keywords (TF-IDF) - TF-IDF bar + `wordcloud` library widget via `st.image`
7. Readability - per-section Flesch bar with color coding + easiest/hardest metrics
8. Similarity - similar pairs table + full cosine similarity heatmap + pyvis network graph

All `adv_` tabs display informational messages if the advanced pipeline stage hasn't run.

Page 5 - Report (`5_report.py`) · Hung

- Download buttons for `final_report.md` and `summary_tables.xlsx`
- Inline preview of first 3000 characters of the Markdown report
- Individual download buttons for each of the 8 chart PNGs

11. Report Generation

Markdown report structure

```
# BeautifulSoup Documentation Analytics - Final Report
1. Dataset Overview          (table: CSV name, row count, description)
2. Scraping Method           (prose: requests, BeautifulSoup4, html.parser
rationale)
3. Extracted Data Summary    (tables: first 5 rows per dataset + link/method
breakdowns)
4. Analysis Results          (table: Q1-Q10 + ADV answers)
5. Charts                    (8 sections with embedded HTTP image links)
6. Key Findings              (6 bullet points with specific numbers)
7. Limitations               (6 bullets)
8. Conclusion                (prose summary)
```

Excel workbook (5 sheets)

Sheet	Content
Sections	Full sections DataFrame (113+ rows × 7 columns)
Links	Full links DataFrame (376+ rows × 4 columns)
Code Examples	Full code_examples DataFrame (220+ rows × 9 columns)
Analytics Summary	Metric/Value pairs for Q1–Q10, link type breakdown, adv_ fields
Top Keywords	keyword/count pairs from Q5

12. Test Suite

Test files and strategies

File	Tests	Strategy
<code>test_collector.py</code>	6	<code>unittest.mock.patch</code> on <code>requests.get</code> . Tests success, HTTP error, connection error, UTF-8 encoding, parent directory creation
<code>test_parser.py</code>	9	Inline fixture HTML (no file required). Tests <code>parse_html()</code> and <code>extract_section_tree()</code> on minimal BS4-style HTML
<code>test_link_classifier.py</code>	25	<code>@pytest.mark.parametrize</code> with 20 specific href/expected pairs + 3 meta-tests verifying all branches reachable and all returns valid
<code>test_extractor.py</code>	12	Inline HTML fixture with all 5 link types. <code>monkeypatch.setattr</code> on CSV output paths

File	Tests	Strategy
<code>test_analyzer.py</code>	12	Synthetic DataFrames. Patches <code>pipeline.analyzer.SUMMARY_STATS_JSON</code> directly (not via <code>shared.constants</code>)
<code>test_api_routes.py</code>	28	<code>FastAPI.testclient.TestClient</code> . Pre-populates <code>data_service._cache</code> directly (bypasses import-time binding issue)
<code>test_pipeline_output.py</code>	28	Real pipeline data. <code>@skip_if_no_data</code> guard skips cleanly before pipeline runs

Total: 120 test cases across 7 files

Critical implementation detail - cache pre-population

`test_api_routes.py` pre-populates `api.services.data_service._cache` directly:

```
import api.services.data_service as ds
ds._cache["sections"] = SECTIONS_DF.copy()
ds._cache["links"]    = LINKS_DF.copy()
ds._cache["code"]     = CODE_DF.copy()
```

Patching `data_service.load_sections` via `unittest.mock.patch` does **not** work because route files import the function by name at module load time:

```
from api.services.data_service import load_sections # bound at import time
```

After import, the name `load_sections` in the route module points directly to the original function - patching the `data_service` module namespace has no effect on the already-bound reference. Pre-populating `_cache` works because `load_sections()` always checks the cache first before calling the real loader.

Running tests

```
# All tests (after pipeline has been run)
pytest tests/ -v

# Only offline tests (no pipeline required)
pytest tests/ -v -k "not pipeline_output"

# With coverage report
pytest tests/ -v --cov=pipeline --cov=api --cov-report=term-missing
```

13. Configuration & Environment

`.env` file (copy from `.env.example`)

```
API_HOST=0.0.0.0
API_PORT=8001
LOG_LEVEL=INFO
API_BASE_URL=http://localhost:8001
SKIP_FETCH=false
```

`app/config.py` reads this file directly by parsing it line-by-line, avoiding any dependency on shell environment variable inheritance. This solves the problem where `os.getenv("API_BASE_URL")` fails in Streamlit because Streamlit pages run as subprocesses that don't inherit the parent terminal's environment.

`config/settings.py`

Uses `pydantic_settings.BaseSettings` which reads from `.env` automatically. Used by the FastAPI side. Fields: `api_host`, `api_port`, `log_level`, `api_base_url`, `skip_fetch`, `http_timeout`.

`requirements.txt` key dependencies

```
# Core pipeline
requests, beautifulsoup4, lxml, pandas>=2.2, numpy>=2.0, matplotlib>=3.9,
openpyxl

# API
fastapi>=0.115, uvicorn[standard], pydantic>=2.7, pydantic-settings>=2.3
httpx, python-multipart, websocket-client

# Frontend
streamlit>=1.36, plotly>=5.22

# Advanced NLP
scikit-learn>=1.5      # TF-IDF, cosine similarity
textstat>=0.7          # Flesch readability
nltk>=3.8.0
networkx>=3.3          # section-link graph
pyvis>=0.3.2           # interactive network graph
wordcloud>=1.9.6       # keyword wordcloud
rapidfuzz>=3.9.0       # fuzzy/Levenshtein search

# Report generation
reportlab>=4.2 (retained), nbconvert, jupyter

# QA
pytest>=8.2, pytest-cov>=5.0, ruff>=0.5
```

Python 3.13 compatibility notes:

- `numpy>=2.0` - 2.x has pre-built Windows wheels for 3.13; `<2.0` caused pip to attempt source builds
 - `wordcloud>=1.9.6` - has 3.13 wheels; `streamlit-wordcloud` (a different package) does not
 - `asyncio.subprocess.__all__` - may be missing in some 3.13 installations due to stdlib corruption; the symptom is Streamlit failing to import asyncio
-

14. Quick-start Guide

Prerequisites

- Python 3.11–3.13
- Git
- Port 8001 available (or configure a different port in `.env`)

Setup

```
# 1. Clone
git clone <repo-url>
cd bs4_analytics

# 2. Virtual environment
python -m venv .venv
.venv\Scripts\activate      # Windows
# source .venv/bin/activate # macOS/Linux

# 3. Install dependencies
pip install -r requirements.txt

# 4. Environment
copy .env.example .env      # Windows
# cp .env.example .env      # macOS/Linux
# Edit .env if port 8001 is taken
```

Running

```
# Run full pipeline (generates all data files, charts, reports)
python -m pipeline.pipeline

# Re-run without re-downloading HTML (faster in development)
python -m pipeline.pipeline --skip-fetch

# Terminal 1 - start API
uvicorn api.main:app --reload --port 8001

# Terminal 2 - start Streamlit
streamlit run app/main.py
```

```
# Swagger UI
# http://localhost:8001/docs

# Streamlit app
# http://localhost:8501
```

Running individual pipeline stages

```
python -m pipeline.collector          # Stage 1: download HTML
python -m pipeline.parser              # Stage 2: parse + count sections
python -m pipeline.extractor          # Stage 3+4: extract sections + links
python -m pipeline.code_extractor     # Stage 5: extract code examples
python -m pipeline.analyzer           # Stage 6: run analytics
python -m pipeline.visualizer         # Stage 7: generate charts
python -m pipeline.reporter           # Stage 8: generate report + XLSX
python -m pipeline.advanced.nlp_analyzer # Advanced: TF-IDF + readability
python -m pipeline.advanced.similarity # Advanced: cosine similarity pairs
python -m pipeline.advanced.graph_builder # Advanced: section-link network
```

Running tests

```
# All tests (run pipeline first)
pytest tests/ -v

# With coverage
pytest tests/ -v --cov=pipeline --cov=api --cov-report=term-missing

# Only tests that don't need pipeline data
pytest tests/ -v -k "not pipeline_output"
```

15. Known Limitations & Design Trade-offs

L1 - "Unknown" section attribution

Content appearing before the first heading - the table of contents, navigation links, language selector links - cannot be attributed to a section by the heading-walk algorithm. These elements are attributed to "Unknown". The Table of Contents section itself accounts for 128 of 376 total links (34%). This is a structural artefact of the Sphinx documentation generator, not an extraction bug.

Mitigation: the `extract_links()` function skips pure pilcrow anchor links (empty link text + bare `# href`) that Sphinx adds as heading permalinks. This removes approximately 100 navigation artefacts.

L2 - "sister" in top keywords

The documentation uses a fictional HTML document about three sisters from Alice in Wonderland throughout all its examples. "sister" ranks 10th in raw frequency (286 occurrences) and is a documentation example data artefact, not a BeautifulSoup concept. TF-IDF partially mitigates this - "sister" scores lower than `tag`, `soup`, and `parser` in the TF-IDF ranking because it is concentrated in example sections rather than spread across the conceptual sections.

L3 - Readability scores on mixed content

`section_text` includes Python code snippets, HTML fragments, and technical identifiers mixed with natural language prose. Code tokens have unusual syllable counts (e.g., "beautifulsoup" = 5 syllables, "lxml" = potentially ambiguous), which artificially deflates Flesch scores. The mean of 55.99 should be interpreted as a lower bound on the true readability.

L4 - Section graph has few edges

The section-link graph contains 112 nodes but few edges because most internal links are navigation anchors (e.g., `#searching-the-tree`) that use kebab-case slugs. The anchor resolution algorithm matches slugs to section titles using three passes, but section titles often contain parentheses, special characters, and Sphinx-specific formatting that prevents exact slug matching. A future improvement would pre-build a slug→title map directly from the heading anchor IDs in the HTML.

L5 - Single-version, single-language source

The analysis covers only the English BS4 documentation at the time of scraping. The source page links to Chinese, Japanese, Korean, Russian, Portuguese, and Spanish translations - none of which are analysed. Version differences between BS3 and BS4 are mentioned in the documentation but not structurally distinguished in the extraction.

L6 - In-memory cache not invalidated between test runs

`data_service._cache` is a module-level dict. In production, it is invalidated by `invalidate_cache()` after each pipeline run. In testing, the `function`-scoped `client` fixture clears and restores the cache around each test. However, if tests are run in a process that has already loaded real pipeline data, the cache may contain real data before the fixture runs. The fixture explicitly calls `ds._cache.clear()` before and after yielding to prevent this.

L7 - Windows asyncio subprocess workaround

The WebSocket implementation uses `sys.executable -m pipeline.pipeline` rather than a plain "python" call, ensuring the subprocess runs inside the correct virtual environment. `ConnectionResetError [WinError 10054]` - raised by Windows when the client disconnects while the subprocess stdout stream is still open - is now caught in a dedicated `except ConnectionResetError` block and suppressed at DEBUG level. A polling fallback on the Home page (`GET /pipeline/status`) activates automatically if the WebSocket path fails, ensuring the Run Pipeline button always works regardless of platform WebSocket support.